

Trusted AI Safety Expert: Module 2

Official Study Guide



Version Number: 20251015

© 2025 Cloud Security Alliance – All Rights Reserved. You may download, store, display on your computer, view, print, and link to the Cloud Security Alliance at <https://cloudsecurityalliance.org> subject to the following: (a) the draft may be used solely for your personal, informational, noncommercial use; (b) the draft may not be modified or altered in any way; (c) the draft may not be redistributed; and (d) the trademark, copyright or other notices may not be removed. You may quote portions of the draft as permitted by the Fair Use provisions of the United States Copyright Act, provided that you attribute the portions to the Cloud Security Alliance.

Acknowledgments

Lead Authors

Tugce Guler
Satwik Kamarthi
Wei Li
Charlie Meyer
Nihar Sanda
Sam Scarpino
James Sheldon
Thulasi Tholeti
Peter Vickers

Contributors

Cansu Canca
Shiran Dudy
Laura Haaber Ihle
Shantanu Jain
Tomo Lazovich
Resmi Ramachandranpillai
Annika Schoene

CSA and NU Global Staff

Marina Bregkou
Josh Buker
Daniele Catteddu
Holly Franklin
Ryan Gifford
Keagan Goodhue
Tara Hanson
Larry Hughes
Nicholas Knopf
Stephen Lumpe
NJ Park
Hannah Rock
Anna Schorr
Stephen Smith
Cathy Jo Swain
Kevin Tobin
Jamie Traynor
Kira Twombly

Table of Contents

- Acknowledgments..... 3
 - Lead Authors..... 3
 - Contributors..... 3
 - CSA and NU Global Staff..... 3
- Table of Contents..... 4
- Introduction to TAISE..... 5
 - Receive a Respected Industry Credential..... 5
 - Acquire In-Demand Skills..... 5
 - Go Beyond Theory..... 5
- Module 2: Generative AI Architecture & Design..... 6**
 - Learning Objectives..... 6
 - 2.2 Module Vocabulary..... 7
 - 2.3 Mathematical Foundations for Machine Learning..... 10
 - 2.4 Understanding AI Architecture Paradigms..... 17
 - 2.5 The Transformer Architecture..... 19
 - 2.6 Attention Mechanisms Deep Dive..... 23
 - 2.7 Pre-training Architecture and Objectives..... 26
 - 2.8 How Generative AI Models Learn..... 29
 - 2.9 Data Ingestion and Storage Strategies..... 32
 - 2.10 Retrieval-Augmented Generation (RAG) Deep Dive..... 37
 - 2.11 Knowledge-Augmented Generation (KAG)..... 42
 - 2.14 Cloud Service Models for Generative AI..... 47
 - 2.15 Deployment Strategies and Hybrid Environments..... 50
 - 2.16 Security and Interoperability..... 52
 - 2.18 Summary..... 57
- Change Log..... 59

Introduction to TAISE

If AI is going to shape the world, we need people who know how to shape it safely.

The **Trusted AI Safety Expert (TAISE)** certificate, created by the Cloud Security Alliance (CSA) and Northeastern University, is a rigorous, research-backed program for professionals who build, manage, or audit intelligent systems. Through 10 comprehensive modules and a final certificate exam, learners gain practical skills to evaluate and mitigate real-world AI risks, apply AI safety and security controls, navigate compliance frameworks, and lead responsible AI adoption across industries. TAISE is more than a certificate—it's a commitment to advancing safe, secure, and responsible AI.

Receive a Respected Industry Credential

TAISE is co-issued by CSA, **the global leader in cloud security standards**, and Northeastern University, **a top academic institution for AI ethics and experiential learning**. This joint recognition enhances your professional credibility, providing a globally respected credential to highlight on resumes, LinkedIn, and professional profiles.

Acquire In-Demand Skills

Professionals will learn to work with frameworks such as **NIST AI RMF, CSA AICM, ISO standards**, and **MITRE A TLAS** while mastering **threat classification, data privacy, model governance, and AI-specific incident response**. These capabilities directly map to the urgent needs of enterprises adopting AI.

Go Beyond Theory

The **TAISE Prompt Library** transforms concepts into hands-on practice. It offers curated prompts and exercises that show learners how to use large language models to reinforce and apply safety principles from each module. While not an exam prep tool, it helps professionals **translate training into job-ready skills**.



Module 2: Generative AI Architecture & Design

Generative AI Architecture and Design

This module explores the technical foundations powering today's AI systems. You'll learn about transformer architecture and attention mechanisms, two of the tools that enable models such as ChatGPT to understand and generate human-like text. You will also dive into the probabilistic denoising diffusion models behind image generation systems such as DALL-E and Stable Diffusion. You'll also gain crucial insights into the architectural decisions behind AI services, exploring cloud deployment strategies, security considerations, and the trade-offs among different service models (IaaS, PaaS, SaaS) that organizations face when implementing generative AI solutions. This module bridges the gap between theoretical understanding and practical implementation, covering everything from how these models learn to fine-tuning processes and advanced techniques such as retrieval-augmented generation (RAG) and knowledge augmented generation (KAG) that enhance AI capabilities within external knowledge sources.



Access the [TAISE Prompt Library](#), a free collection of curated prompts designed to complement the TAISE training. This optional resource provides example prompts and techniques that show how to use large language models (LLMs) to explore, reinforce, and apply key ideas from each module.

Learning Objectives

In this module, you will learn to:

- Understand the fundamental architecture of transformers, attention mechanisms, and probabilistic denoising diffusion models
- Analyze how generative AI models learn through training data and fine-tuning
- Understand how RAG supplements LLMs with information retrieval systems
- Explore data ingestion (where data comes from) and data storing
- Conceptualize knowledge augmented generation (KAG)
- Distinguish between generative AI model architectures and service architectures
- Evaluate and select appropriate cloud service models (IaaS, PaaS, SaaS) for AI implementations
- Design hybrid cloud architectures that balance performance, security, and compliance
- Conceptualize secure API integrations for AI models with proper authentication and protection

2.2 Module Vocabulary

The evolution of neural network architectures has transformed how machines understand and generate data, progressing from early **recurrent neural networks (RNNs)** that processed sequential information step by step to revolutionary **transformer models** that leverage self-attention mechanisms to capture complex relationships across entire sequences simultaneously. This architectural leap enabled the development of powerful systems such as **generative pre-trained transformers (GPTs)**, which exemplify how modern **generative models** can learn underlying data patterns to create remarkably human-like content.

At the heart of these systems lies sophisticated text processing, where input is broken down into **tokens**—the fundamental units that models manipulate—and converted into numerical **embeddings** that capture semantic meaning. The transformer's attention mechanism orchestrates this process through specialized components: **queries** that seek information, **keys** that advertise available content, and **values** that provide actual data, all coordinated by **attention heads** that develop distinct linguistic expertise during training. These architectures incorporate **inductive biases**—built-in assumptions about information flow—and are trained using approaches such as **masked language modeling (MLM)** for bidirectional understanding or **causal language modeling (CLM)** for sequential prediction.

Beyond text, generative AI encompasses diverse approaches to synthetic content creation. **Variational autoencoders (VAEs)** use probabilistic encoding to generate new samples from compressed **latent spaces**, while **generative adversarial networks (GANs)** employ competing networks in an adversarial game to produce highly realistic outputs. More recently, **diffusion models** have emerged as a powerful alternative, creating content by gradually transforming random noise into structured data, with innovations such as **denoising diffusion implicit models (DDIM)** dramatically accelerating this process. Together, these technologies represent a comprehensive tool kit for understanding, processing, and generating diverse forms of synthetic content across multiple domains.

Below are the vocabulary terms for this module. Click on the **+** to see each definition.

An **attention head** is a specialized component of a transformer architecture that analyzes text from a specific perspective, like an expert focusing on one aspect of language. Each attention head naturally develops its own specialty during training: One might become an expert at tracking grammatical relationships, another at understanding pronoun references, and another at recognizing semantic similarities between words.

Causal language modeling (CLM) is a training approach, used in transformer models, that predicts the next word given all previous words, processing text sequentially from left to right. This method requires decoder-only architectures with masked self-attention, where each token can only see backwards in the sequence, never forwards.

Denoising diffusion implicit models (DDIM) are an advancement over standard diffusion models that speed up inference by using deterministic sampling, reducing the number of denoising steps needed. This improvement applies at inference time and does not require retraining the original diffusion model, making it especially useful when retraining would be costly.

A machine learning model compresses the raw numerical representation of an input object, typically a high-dimensional vector or matrix, into a typically low-dimensional **embedding vector** by a series of transformations (see Introduction). The low-dimensional space in which the embedding vector lies is called the **latent space**.

Diffusion models are generative models that create data (such as images, audio, or text) by starting from random noise and gradually denoising it through a sequence of learned steps, effectively reversing a simulated noising process. While they can produce high-quality outputs, standard diffusion models often require hundreds or thousands of denoising steps, making generation slow.

A **generative model** is a type of machine learning model that learns the underlying distribution of data so it can create new, similar data samples—such as generating text, images, audio, and synthetic proteins.

Generative adversarial networks (GANs) are a generative neural network architecture, which emerged in 2014, that uses an adversarial game between two networks: a generator that creates synthetic images and a discriminator that tries to distinguish real from synthetic images. They can also be applied beyond images, generating data such as text, audio, and protein sequences.

Generative pre-trained transformer (GPT) is a type of generative large language model that uses the transformer architecture to generate human-like text. It is first pre-trained on massive amounts of text data to learn grammar, facts, and patterns in language, and can then be fine-tuned for specific tasks such as summarization, translation, or question answering.

Inductive biases are built-in assumptions about how information should flow within neural network architectures. These biases impose specific structural constraints or assumptions, such as spatial relationships in images or sequential patterns in text.

A **key** is one of the three components in the attention mechanism (along with queries and values). The key is a numerical vector that represents "what type of information do I advertise having?"

Latent space is the low-dimensional space in which embedding vectors lie. A machine learning model compresses the raw numerical representation of an input object, typically a high dimensional vector or matrix, into a typically low-dimensional embedding vector by a series of transformations.

Masked language modeling (MLM) is a self-supervised training approach, used in transformer models, that works like a fill-in-the-blank exercise, where the model randomly masks tokens throughout a sentence and learns to predict them using context from both directions.

A **query** refers to one of the three components in the attention mechanism (along with keys and values). The query is a numerical vector that represents "what information am I looking for?"

A **recurrent neural network (RNN)** is a neural network architecture designed to process sequential data (sequence of elements) by maintaining a hidden memory state that gets updated as the network processes the sequence elements one after the other. At a given time step, the memory, containing a summary of all previous elements, is updated with the information in the current element. The output for the current element is computed directly from the updated memory, giving a contextualized prediction with information from previous elements.

A **token** is the basic unit of text that a sequence model processes, created by breaking down input text into smaller pieces that don't always correspond directly to words. This process is called tokenization. For

example, the sentence "It is raining outside" might be broken into six tokens: "It," "is," "rain," "ing," "out," and "side."

Each token is represented as a learnable numerical vector, referred to as its **embedding**, as it is processed by the model. The embedding of a token is modified by each layer of the model. The initial embedding of a token, fed as input to the first layer, is non-contextualized in that it does not carry any information from the other tokens in the sequence. Each layer adds contextual information to a token's embedding.

Transformer models are a neural network architecture introduced in 2017. These models process input data using self-attention mechanisms, allowing the model to capture long-range relationships between all elements in an input sequence simultaneously. They are widely used in natural language processing, vision, and multimodal tasks, replacing recurrent and convolutional approaches in many domains.

A **value** is one of the three components in the attention mechanism (along with queries and keys). The value is a numerical vector that represents "what actual information do I provide when asked?"

Variational autoencoders (VAEs) are a probabilistic approach for image generation that encodes images into a compressed latent space and decodes them back to generate new images. A VAE has two main parts: an encoder that inputs data into a simplified, low-dimensional representation (latent space) and a decoder that reconstructs the original data from this compressed representation. VAEs introduce a probabilistic approach by encoding inputs into probability distributions, and this allows them to create diverse yet realistic synthetic outputs.

2.3 Mathematical Foundations for Machine Learning

Mathematical Objects and Operations for Machine learning

Vectors and matrices are the core mathematical objects used in machine learning that allow representing things or objects (images, text, proteins, persons, categories) in the real world and are also used to define model parameters (described later). This representation allows machine learning models to manipulate these objects and extract useful patterns from the data.

A **vector** is simply a list of numbers. One can think of it as a row or a column in a table or spreadsheet. A **matrix** is a rectangular grid of numbers arranged in rows and columns. One can think of it as a table or a spreadsheet of numbers. A vector can be thought of as a special case of a matrix with only one row or column.

The **length or dimension** of a vector is the number of entries in it. The **shape or dimension** of a matrix is given by the number of rows and columns in it; for example, a 100×20 –dimensional matrix has 100 rows and 20 columns. Typically, vectors are denoted by small letters (e.g., x , u , v) and matrices by capital letters (e.g., A , U , V).

In machine learning, vectors and matrices are used to:

- **Represent objects:** For example, if you want to describe a person's height, weight, and age, you can write those three numbers side by side. This may serve as a representation of a person for a disease study. A greyscale image is represented as a matrix containing the intensity of each pixel arranged in two dimensions.
- **Define model parameters:** The model parameters are defined to interact and manipulate the raw representations of the objects and convert them into higher-level representations (more useful representations), which are referred to as **embeddings**.

Common operations used for manipulating the object representations include **dot product**, **matrix-vector** product, and **matrix-matrix** product. A **dot product** is defined between two vectors of the same length. It is computed by multiplying the corresponding elements of the two vectors at the same position and summing over the products, giving a single number. In other words, the dot product between vectors u and v is given by $u \cdot v = \sum u[i]v[i]$ where $u[i]$ and $v[i]$ are the entries in u and v at position i . Often, a dot product serves to measure the similarity between two vectors; a higher dot product implies a higher similarity (similar values at each position).

A **matrix-matrix product** is a product between two matrices where the number of columns in the first matrix is the same as the number of rows in another matrix. The result is another matrix with the number of rows in the first matrix and the number of columns in the other. It is obtained by taking the dot products between row vectors from the first matrix and column vectors from the other. Mathematically, for matrices A and B of shape 5×3 and 3×7 , $P[i, j] = \sum a_i \cdot b_j$ gives the entry at row i and column j of the matrix-matrix product $P = AB$, where a_i and b_j is the row i and column j in matrix A and B , respectively. The shape of P is 5×7 .

A **matrix-vector product** is defined between a matrix and a vector. One can think of it as a special case of matrix-matrix product where the second matrix is actually a vector of dimension $d \times 1$, where d is the

length of the vector. The result of a matrix-vector product is another vector with the same length as the number of rows in the matrix.

What is a machine learning model?

A **machine learning model** is like a smart transformation or function that takes raw input data—images, text, numbers, or protein sequences—and transforms that data into useful outputs specific to a task, such as recognizing faces, translating languages, or predicting trends. The way this function works depends on its **architecture**, which you can think of as the model's blueprint or structure. This blueprint defines how the input and output relate to each other, shaping the form of that relationship.

Inside the model, there are many adjustable numeric **parameters**—internal variables that specify exactly how the inputs are turned into outputs. But before the model can do a good job, these parameters need to be set just right. The model parameters are learned during the **training** phase by fitting to relevant data, referred to as the training data. After the parameters are learned, the model can be used for **inference**—predicting outputs on new data.

How are models trained?

Through an iterative process, the model adjusts its parameters step by step to reduce what is called the **loss**, computed with the training data and the model's output on the training data. The form of the **loss function** used to compute the loss could depend on the data and the prediction task or the pattern in the data that we want to capture. Common choices of loss function include cross-entropy loss and mean squared error loss.

More generally, one can think of the loss as a quantity capturing how poorly the model captures useful patterns in the data, with a lower value indicating a better model fit. In the simplest setting, referred to as **supervised learning**, the training data consists of many **examples** of an **input** (e.g., image) along with the correct **output** or **outputs** (e.g., objects in the image), referred to as the **targets** or **labels**. Parameters are adjusted step by step to reduce the errors between the targets and the model's predictions—captured by the mean squared error loss. After many such adjustments to reduce the loss, the parameters settle into optimal values, capturing the desired patterns within the data.

Gradient descent: The mathematics underneath this training procedure requires computing the **gradient** of the loss function with respect to the parameters, which captures the direction in which the parameter vector (all parameters put together in a list) should move to decrease the value of the loss function. This algorithm is referred to as **gradient descent**. For computationally efficient training and robust performance, a variant of gradient descent referred to as **stochastic gradient descent** is used. In stochastic gradient descent, the gradient is estimated on a loss computed on a random subset of the data at each iteration, referred to as the **batch**, instead of using the full dataset. Modern deep learning models are trained with variants of stochastic gradient descent, such as the **Adam (adaptive moment estimation) algorithm** for an even more robust training.

The term “deep” in deep learning signifies the **multi-layered architecture** of the model, where outputs from one layer feed into the next layer as inputs, with each layer having its own set of parameters. The outputs from the last layer serve as the final outputs or predictions on which the loss function is computed. A common parameter in all deep learning architectures is a layer-wise **weight matrix**, which is multiplied with the layer's input via matrix multiplication. The layer's outputs are computed from the nonlinear (e.g., softmax, sigmoid, ReLU) functions applied to the resultant product with many variations across architectures. The multilayered architectural choice is the key to its enhanced performance

compared to simpler models, such as shallow neural networks, logistic regression, and support vector machines.

Backpropagation algorithm: However, this leads to additional challenges in stochastic gradient descent-based parameter learning. Computing the gradients for each layer's parameters involves multiplying the gradients from higher layers. This is performed efficiently via the backpropagation algorithm, where the gradients for the highest layer, closest to the loss function, are computed and then iteratively multiplied with the gradients of the next layer below it. This chain of multiplications can cause the gradients for lower layers to either shrink exponentially, leading to **vanishing gradients**, or grow exponentially, resulting in **exploding gradients**.

Both phenomena cause significant challenges during training: Vanishing gradients slow down learning by producing negligible updates for early layers, while exploding gradients can cause numerical instability and erratic parameter updates. Notably, these issues arise from the spectral properties of the weight matrix at each layer and can coexist within the same network, affecting different components of the gradient differently. Several approaches and architectural choices have been developed to mitigate the issue of vanishing and exploding gradients.

Embedding and latent space: A deep learning network processes objects—such as words, images, or sensor readings—by representing them as vectors (or sometimes matrices) and repeatedly transforming these representations with learned parameters. The raw or initial representation of an object, before applying any transformation, is referred to as an **encoding** vector or matrix.

Each transformation typically involves a **matrix-vector or matrix-matrix product** between the learned parameter matrix (or weight matrix) and the object's current representation. The result is a new **embedding**—a numerical representation capturing the object's features in a way the model can use. After this linear transformation, the network applies a **nonlinear activation function** (e.g., [sigmoid](#), [ReLU](#)) to each entry. This step is crucial as it allows the network to model complex, nonlinear relationships in the data that pure matrix multiplication alone could not capture.

By stacking many such layers, each producing progressively refined embeddings, deep networks can learn increasingly abstract and useful representations. Remarkably, this simple cycle—linear transformation followed by nonlinearity—proves powerful in shaping embeddings that are highly effective for a wide variety of tasks. Typically, the dimension of an embedding vector is much lower than the raw representation vector. Thus, the embedding vectors serve as a compressed representation of the object. The compressed representation promotes preserving important patterns in the data and removing the noise. The space in which the embedding vector lies is called the **latent space**. Broadly, latent space could mean R^d , the d dimensional real number space, where d is the length of the embedding vector. Often, however, it is used to describe the manifold or the region in R^d where the training example embeddings lie.

Learning Paradigms

In **supervised learning**, each training example has a separate input part (e.g., an image) and a target part (e.g., labels for objects in the image: cat, dog, car, etc). Such data is referred to as the **labeled data**. The most common types of **supervised learning** tasks include **classification**: predicting a discrete target (e.g., "cat," "dog," "car" from an image) and **regression**: predicting a discrete target (e.g.,

temperature or precipitation in weather modeling). In **unsupervised learning** (e.g., clustering or dimensionality reduction), a training example only has the input part, without a target. Such data is referred to as **unlabeled data**. Typically, for most applications, unlabeled data is much easier to acquire and often available in larger quantities (e.g., all images uploaded on Instagram).

Most modern AI tools are designed to exploit the patterns in the unlabeled data, in addition to that in the labeled data. This approach has resulted in vast performance improvements compared to purely supervised approaches. In **semi-supervised** learning, some examples in the training data have both an input and a target, whereas some examples only have input. In **self-supervised** learning, although the training examples do not have a target (similar to unsupervised learning), targets are created from the inputs. Modern advances in generative AI and foundation models heavily rely on self-supervised learning.

In traditional machine learning, once a model is trained, the parameters stay fixed. The model then uses this learned knowledge to make predictions on new, unseen data. However, modern machine learning often takes this a step further. **Foundation models** are trained on massive unlabeled datasets (e.g., all books ever written, all images on Instagram). This process is called **pre-training**. Next, the model is **fine-tuned** for any downstream task (e.g., question-and-answer task, scene summarization), where the pretrained model's parameters are adjusted using relevant labeled data for the task. Fine-tuning usually requires much less new data because the model already understands many useful patterns from training on the unlabeled data.

Data Types

Tabular data consists of rows and columns, similar to a spreadsheet. Each column represents a particular **feature, variable, or attribute**, and each row represents a single **example** or **observation**, with values for all features. In tabular data, features are often heterogeneous: Some may be numerical (e.g., age, income), others categorical (e.g., country, product type), and others binary (e.g., has_disease: yes/no). Each feature is interpreted independently in terms of meaning, although they can interact statistically in complex ways that models must learn. Changing the order of the columns has no effect on the data as it still represents the same examples.

Tabular data is almost always fixed-dimensional: Each row has the same set of columns. This makes it naturally compatible with traditional machine learning algorithms such as logistic regression, random forests, support vector machines (SVM), gradient boosted trees, and shallow or deep multilayer perceptrons (MLPs). Popular examples of tabular data include customer profiles in a CRM system, patient health records, credit scoring datasets, sensor measurements from industrial equipment, and results from scientific experiments.

Sequential data (e.g., text) consists of **elements or tokens** (e.g., words) that are ordered. The ordering is important in that changing the order of the words can change the meaning or the interpretation of the text. Furthermore, the same word can have different meanings based on its **context** (words that come before and after it). Thus, a word not only contains information on how it should be interpreted but also on how other words should be interpreted. Architectures of machine learning models for sequential data, such as recurrent neural networks (RNNs), long short-term memory (LSTM), and transformers, are explicitly designed to extract this contextual information and capture the influence one word has on the meaning of another.

Typically, sequential data has variable length. One piece of text can have 1,000 words, and another can have 10,000, yet both are expected to be processed by the same model. Traditional machine learning models, such as multilayer perceptron (MLP) or a fully connected deep network, random forest, support vector machine (SVM), or gradient boosted trees, are not equipped to handle variable length data inputs. Popular examples of sequential data include text, time series (e.g., stock prices), speech, or video and biological sequences such as proteins, DNA, and RNA.

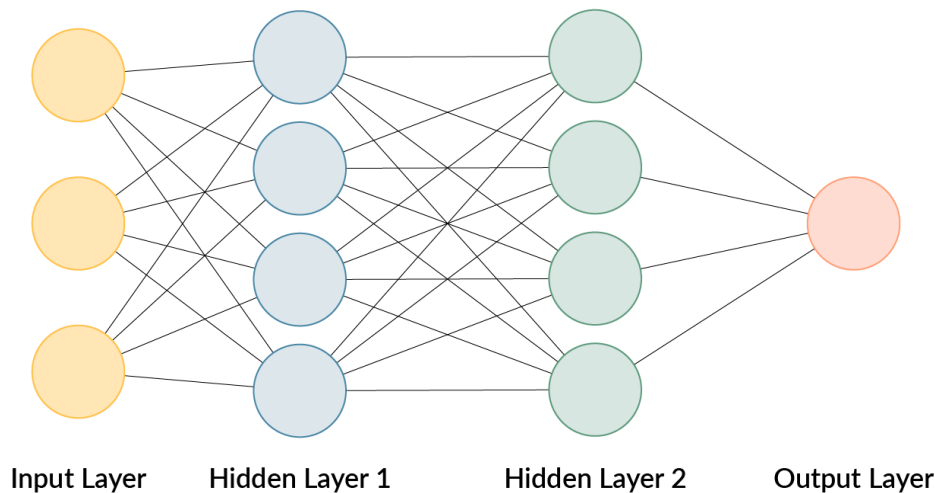
Image data consists of pixels arranged in a fixed spatial grid, typically in two dimensions (height × width) with one or more **channels** (e.g., grayscale has one channel, RGB color has three). Each pixel stores an intensity or color value. Unlike tabular data, the position of a pixel is essential—shuffling pixel columns or rows destroys the image’s structure and meaning. Images have strong spatial dependencies: Neighboring pixels are often highly correlated, and patterns such as edges, shapes, and textures emerge from local arrangements of pixel values. Context matters because the interpretation of a pixel depends on the broader region it belongs to (for example, a pixel that is “blue” could be part of the sky, clothing, or water, depending on surrounding pixels).

Machine learning architectures for image data—such as convolutional neural networks (CNNs), vision transformers (ViTs), and hybrid models—are explicitly designed to exploit spatial locality and hierarchical patterns in images. While an image is naturally represented as a 2D grid of pixels, vision transformers treat them as sequences of patches (e.g., 16×16 pixels) extracted from the image, where the ordering of patches corresponds to their original position in the image. Popular examples of image data include medical scans (e.g., X-rays, MRIs), satellite imagery, photographs for object recognition, handwriting in digit classification, and microscopic images in biological research.

Traditional Deep Learning Architectures

A **multilayer perceptron (MLP)** is a neural network architecture composed of multiple layers of interconnected neurons, where each neuron in one layer is connected to every neuron in the next layer. An MLP processes tabular input data (each input has the same dimension) by passing it through a sequence of layers, each performing a linear transformation (multiplication with a matrix) followed by a nonlinear activation function (e.g., sigmoid, softmax, and ReLU). At each layer, the input is transformed into a higher-level feature representation, allowing the model to capture intricate patterns and relationships.

This layered structure enables the network to learn complex dependencies among the input features. MLPs lack mechanisms to explicitly model temporal or spatial structure, and they are generally less effective on sequential data compared to architectures such as RNNs or transformers. Despite this, MLPs remain foundational in machine learning due to their simplicity and effectiveness on a wide range of non-sequential tasks. An MLP belongs to a broader class of networks without cycles or feedback loops, referred to as **feedforward networks**, capable of processing an input in a parallel fashion.



Multilayer Perceptron

<Alternative text: A simple diagram of a multilayer perceptron. Nodes make up an input layer, which feeds into nodes in two hidden layers before converging at one node in the output layer.>

A **recurrent neural network (RNN)** is a neural networks architecture designed to process sequential data (sequence of elements) by maintaining a hidden memory state that gets updated as the network processes the sequence elements one after the other. At a given time step, the memory, containing a summary of all previous elements, is updated with the information in the current element. The output for the current element is computed directly from the updated memory, giving a contextualized prediction with information from previous elements. An RNN has a feedback loop due to the manner in which hidden states are updated, requiring slower sequential processing of the sequence elements, unlike feedforward networks that process elements in parallel.

A powerful **deep RNN** can be obtained by stacking multiple RNN layers, where the output from one layer is fed as input to the next. Although the standard RNNs can only capture past dependencies from previous elements, **bidirectional RNNs (BiRNNs)** process the sequence in both forward and backward directions, enabling the model to capture context from past and future elements simultaneously.

Although RNN architectures are designed to capture long-term dependencies between distant elements in a sequence, they often struggle to do so effectively in practice due to the vanishing and exploding gradient problems during backpropagation. Because of these challenges, RNNs have largely fallen out of favor in modern machine learning, being increasingly replaced by transformer-based models that better capture long-range dependencies.

A **convolutional neural network (CNN)** is a neural network architecture designed to process data with a grid-like structure, such as images (2D grids of pixels) or time series (1D sequences). CNNs use convolutional layers that apply kernel filters with learnable parameters to process fixed-size (e.g., 5×5 pixels) patches of an image. These filters slide across the spatial dimensions of the image to detect local patterns such as edges, textures, or more complex features. Each convolutional layer transforms the input into a set of feature maps (transformations), preserving spatial relationships by capturing the patterns'

positions within the input. Pooling layers are often interleaved with convolutions to downsample the feature maps, reducing spatial dimensions while retaining the most important information. Deeper layers combine lower-level features into more abstract and complex patterns, enabling the network to learn hierarchical representations.

Unlike fully connected networks, CNNs share parameters across spatial locations, making them more efficient and less prone to overfitting for high-dimensional inputs such as images. A powerful deep CNN can be obtained by stacking multiple convolutional and pooling layers, followed by fully connected layers for classification or regression. CNNs process the entire input in parallel, making them highly efficient for modern hardware like GPUs. They are among the most widely used architectures in computer vision, and with modern advances such as residual connections and batch normalization, very deep CNNs can be trained effectively without suffering severely from the vanishing or exploding gradient problem. Although CNNs remain widely used in computer vision tasks, they are increasingly being replaced by vision transformers because CNNs are not inherently designed to capture long-range dependencies effectively.

2.4 Understanding AI Architecture Paradigms

The Two Faces of AI Architecture

The term "AI architecture" encompasses two fundamentally different perspectives that are essential to understand for anyone involved in planning, building, or managing AI systems in enterprise environments.

When architects, data scientists, and business leaders discuss AI architecture, they may refer to entirely different aspects of the same system. This ambiguity can lead to miscommunication and suboptimal decisions. Let us break down the two types:

1. **Model Architecture:** This comprises the internal structure and design of AI models, including their neural network layers, parameters, task-specific loss functions, training methodologies, and optimization techniques. The architecture determines how a model processes the input and generates outputs.
2. **Services Architecture:** This is the external system design that delivers AI capabilities to users, encompassing API gateways, load balancers, caching systems, monitoring infrastructure, and components enabling reliable, scalable access to AI functionality.

Services Architecture: The External Perspective

Services architecture focuses on how AI capabilities are packaged, delivered, and consumed. This treats the AI model as a computational resource that must be made available reliably, securely, and efficiently.

The API gateway serves as the primary entry point, handling request routing, authentication, rate limiting, and protocol translation. Load balancing is challenging due to the stateful nature of many AI interactions and significant computational requirements.

Caching strategies operate at multiple levels, from response caching for identical queries to sophisticated prompt caching. The stochastic nature of generative AI models complicates caching decisions, as identical inputs may produce different outputs.

Example: The OpenAI API employs rate-limited endpoints, distributed model serving, intelligent request routing based on availability and user tier, comprehensive monitoring, and robust error handling with graceful degradation.

Key Architectural Differences

Aspect	Model Architecture	Services Architecture
Focus	Internal model design	External system design
Concerns	Data accuracy, efficiency, training performance, security	Scalability, reliability, security
Stakeholders	ML engineers, researchers	System architects, DevOps
Optimization	Model performance, inference speed	System throughput, availability
Evolution	New model versions	Service updates, scaling



Key Takeaways

- Model and services architectures address different concerns requiring different expertise
- Model architecture focuses on internal design and accuracy
- Services architecture emphasizes external delivery and scalability
- Understanding both perspectives is essential for successful AI implementation
- Different stakeholders naturally gravitate toward different architectural perspectives

2.5 The Transformer Architecture

Understanding Transformer Architecture

The transformer architecture, introduced in the landmark paper "[Attention is All You Need](#)" (2017), fundamentally changed how we approach sequence processing in AI. When we model sequential phenomena such as language, we need to account for relationships between elements that are far apart in the sequence. Simple deep network architectures such as multilayer perceptron (MLP) are beholden to fixed positional relationships—word 1 relates to word 10 in a fixed manner. Consider these two sentences: "The dog chased the cat" and "The wild dog chased the black cat." The word "cat" should ideally receive similar contextual information from the words "dog" and "chased" since the core meaning and relationships are nearly identical.

However, in an MLP, since the positions of these words are not identical across the two sentences (dog: 2 vs. 3, chased: 3 vs. 4, cat: 5 vs. 7, etc.), completely different parameters would be used to compute these relationships. This means the model might infer different contextual information for the word "cat" in each sentence, despite the semantic relationships being essentially the same—a fundamental limitation that prevents effective learning of similar linguistic patterns appearing at different positions.

Sequential architectures such as recurrent neural networks (RNNs) attempted to solve this positional rigidity by processing words sequentially, one at a time, while maintaining a hidden state that carries information forward through the sequence. Unlike MLPs, RNNs use the same set of parameters at each time step to update the hidden memory state from each word, meaning the relationship between any word and the next word is computed using identical weights, regardless of their absolute positions in the sentence. The RNN's hidden state accumulates contextual information as it moves through the sequence, so "cat" receives context from all previously processed words through this evolving memory.

This approach allows RNNs to learn similar linguistic patterns at different positions better than an MLP. In a standard RNN, a word can only incorporate the context from the previously appearing words. However, the context from future words is also useful when interpreting a sentence. For example, in the sentence "Apple announced record profits despite the recession," we can't determine whether "Apple" refers to the fruit or the company until we see the future words "announced" and "profits." In some tasks, such as **machine translation, named entity recognition, and sentiment analysis**, it is feasible to incorporate future context since we have access to the complete sentence during processing, though standard RNN doesn't allow it.

BiRNN does address this limitation by processing the sequence in both directions—one RNN processes from left to right (past to future), while another processes from right to left (future to past). The final representation for each word combines information from both directions, allowing each word to benefit from both preceding and following context. Despite RNNs and BiRNNs improving over MLPs by enabling contextual information flow, they face significant practical limitations in capturing long-range dependencies. The vanishing and exploding gradient problems make it difficult to learn relationships between distant words during training.

Additionally, RNNs suffer from a fundamental bottleneck: All contextual information must be compressed into fixed-size hidden states, leading to information loss as sequences grow longer. Consider the sentence "The cat that the neighbor's dog from down the street chased yesterday was actually quite brave." When processing "brave," the RNN needs information about "cat" from the beginning, but this information has been diluted through 15 intermediate processing steps and compressed repeatedly into the same hidden state dimensions. Unlike direct connections, RNNs cannot directly access context from distant words—information must pass sequentially through all intermediate states, making long-range dependencies increasingly difficult to maintain. Finally, BiRNNs require sequential processing in both directions, making them slower to train compared to architectures that can process all positions in parallel. Because of these challenges, RNNs have largely fallen out of favor in modern machine learning, being increasingly replaced by transformer-based models that better capture long-range dependencies.

Transformers overcome these limitations by computing relationships dynamically through an attention mechanism: Rather than forcing all context through memory bottlenecks, any word can look at any other word using the same attention parameters, regardless of their absolute positions in the sequence. While transformers do require explicit positional encodings to inform the model about token positions (since attention itself is position-agnostic), the key breakthrough is that the same attention mechanism can connect any pair of words—whether they're adjacent or separated by dozens of tokens. It's like switching from assigned airplane seats to open seating—everyone can talk to everyone else using the same conversation rules, with just a name tag (positional encoding) indicating where they're sitting. This parallel processing not only makes training much faster but also allows the model to capture long-range dependencies that RNNs struggle with, fundamentally changing what's possible in natural language processing.

Key Components of Transformers

At its core, a transformer is about tokens talking to each other. A token is a basic unit of text that a model processes. When you input text, it is broken down into tokens through a process called tokenization so it can be processed by the model. For example, the sentence "It is raining outside" might be broken into six tokens: "It," "is," "rain," "ing," "out," and "side." (For the remaining portion of the module, we will use the term "token" and "word" interchangeably.) Imagine each word in a sentence as a person in a room who needs to understand what everyone else is saying to determine their own role. This is fundamentally what attention does—it lets every token look at every other token and decide which ones matter most for its current representation.

Token-to-Token Interactions (Attention)

At the heart of the transformer lies a simple but powerful idea: Let every word in a sentence directly communicate with every other word, and condition those communication lines on their content. Sequential models, despite their improvements, were still like a game of telephone, where word 1 had to pass information through words 2, 3, and 4 to reach word 5. By the time the message arrived, it was often distorted or lost. Transformers eliminate the middlemen—every word can directly ask every other word, "How relevant are you to understanding what I mean?" When processing entire input sequences, the

model stacks individual query, key, and value vectors into matrices, allowing parallel computation across tokens.

This direct communication happens through attention scores. In the sentence "Apple announced record profits despite the recession," when the word "Apple" is processed, it sends out a query to all other words: "I need context to know if I'm a fruit or a company—who has information that can help?" The words "announced" respond strongly, "profit" and "recession" provide different but also relevant information, whereas "record," "despite," and "the" contain minimal information. "Apple" attends to each of these words proportional to their relevance, quantified by a relevance score. These relevance scores become weights that determine how much information flows from each word to "Apple." It's like each word gets to conduct a survey of all other words, then it creates a custom blend of their responses to better understand its own role.

This happens for every word simultaneously. While "Apple" is figuring out its meaning by polling other words, "despite" is doing the same thing—perhaps paying special attention to "profit" and "recession." This massive parallel conversation, with every word talking to every other word at once, creates a rich web of understanding that captures both local patterns (adjacent words) and long-distance relationships (words that are far apart but conceptually connected). We'll see later how this is implemented through query, key, and value matrices, but the core insight is beautifully simple: Understanding emerges from letting every part of the input directly interact with every other part.

Multiple Attention Heads—Different Perspectives

Think of attention heads as a panel of experts analyzing the same sentence from different angles. Just as a film critic might focus on cinematography while another examines the plot, each attention head develops its own specialty. One head might become an expert at tracking grammatical dependencies—who did what to whom. Another might specialize in coreference resolution—understanding that "it" in paragraph three refers to "the proposal" in paragraph one. A third might excel at semantic similarity—recognizing that "automobile" and "car" mean the same thing.

It's important to note that these heads are not explicitly told what to look for; they learn patterns during training. We don't assign one to be the "grammar head" or another to be the "meaning head." Instead, through training on vast amounts of text, they naturally specialize. It's like how a group of detectives investigating the same case might naturally divide the work—one notices timeline inconsistencies, another spots behavioral patterns, a third sees financial motives. With dozens or even hundreds of heads working in parallel (modern large models can have 32 to over 100 attention heads per layer), the transformer builds a rich, multi-faceted understanding of each sentence. What one head misses, another catches.

Stacking Layers—Building Understanding

Transformers gain their deep understanding by stacking multiple layers, each building on the insights of the previous one. It's like constructing a building—you can't install windows before you have walls, and you can't have walls without a foundation. Each layer adds a new level of sophistication to the model's understanding.

Research has revealed a fascinating progression through these layers. Early layers act like careful readers, parsing the basic structure—which words modify which others, where sentences begin and end. Middle layers start to see meaning, connecting "dog" to "puppy" and understanding that "she" refers to "the scientist" as mentioned earlier. By the final layers, the model approximates abstract concepts, understanding not just what words mean but what the text is trying to accomplish: Is this an argument? A story? A question seeking specific information?

This layered architecture mirrors how we might analyze a complex text. First, we parse the words, then we understand the sentences, then we grasp the paragraphs, and finally, we comprehend the author's intent. But while we do this sequentially, transformers do it all at once, with each layer simultaneously refining its understanding based on the enriched representations from below. The depth isn't just about having more parameters—it's about building understanding layer by layer, from surface patterns to deep meaning.



Pause and Reflect: Think about a sentence like "The bank by the river bank closed." How might multiple attention heads help a model understand the different meanings of "bank" in this context?

2.6 Attention Mechanisms Deep Dive

How Attention Works

Now that we've seen how transformers enable words to talk to each other, let's look at the mechanism that makes these connections possible.

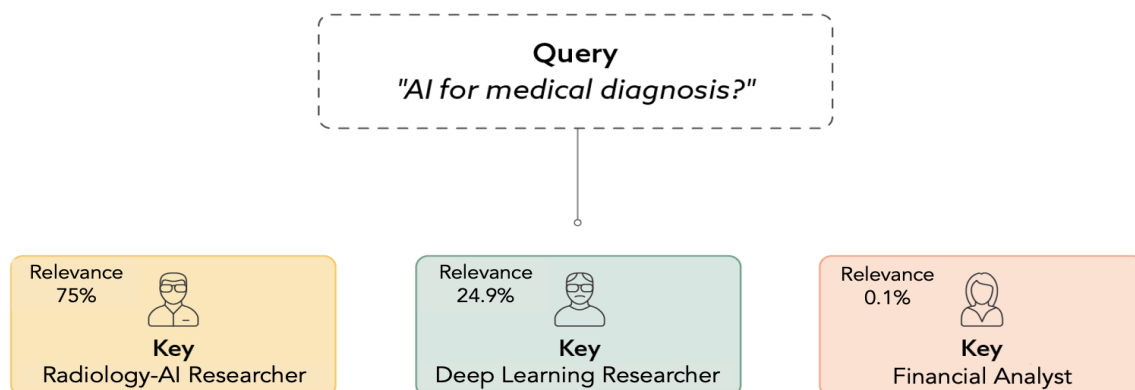
Traditional neural architectures come with built-in assumptions, which the community calls "inductive biases" about how information should flow. Convolutional neural networks assume pixels should mainly talk to their neighbors. Recurrent networks assume words should pass information forward in sequence. But what if we let the model discover these patterns for itself? What if, instead of prescribing who talks to whom, we let the model learn these relationships from data? This is the radical idea behind attention.

Who Am I to You? Queries and Keys

Let's build up the attention mechanism step by step, starting with just two of its three components: queries and keys.

Imagine you're at a massive conference with thousands of attendees. You're looking for people who can help with your specific project—say, machine learning for healthcare. You walk around with a question in mind (your query): "Who works on AI for medical diagnosis?" Meanwhile, everyone else wears badges listing their expertise (their keys): "Deep Learning Researcher," "Radiology-AI Researcher," "Financial Analyst."

You compare your question to everyone's badges. Some matches are the best fit—the Radiologist-AI Researcher may be the most relevant to your query. Others are somewhat relevant—the Deep Learning Researcher might have relevant technical knowledge. Some have no relevance at all—the Financial Analyst probably can't help. The relevance between the query and keys determine who you'll spend time talking to.



<Alternative text: Three badges representing the keys "Radiologist-AI Researcher," "Deep Learning Researcher," and "Financial Analyst." The query "Who works on AI for medical diagnosis?" has the most relevance with Radiologist-AI Researcher, some relevance with Deep Learning Researcher, and no relevance with Financial Analyst.>

This is how attention works. Every word generates both a query ("What information do I need?") and a key ("What information do I have to offer?"). Instead of human interpretable text, used for illustration purposes here, each query and key in the attention mechanism is a numerical vector. The model computes compatibility between every query and every key using a vector dot product—a mathematical way of measuring how well two vectors align. High dot products mean strong relevance; low dot products mean little relevance.

The Complete Picture: Adding Values

Here's where attention becomes truly clever. Knowing who to pay attention to is only half the challenge—we also need to know what information to exchange. This is where values come in.

Let's return to our conference. You've identified the relevant people (using query-key similarity-based attention), but now you need to actually learn from them. The value is what each person actually tells you when you talk to them. Crucially, what someone advertises (their key) might differ from what they share (their value). The radiologist's badge might say "Medical Imaging Expert" (key), but their actual advice might be "Convolutional neural networks (CNNs) work great for tumor detection, but watch out for data imbalance" (value).

In transformer terms:

- **Query:** What information am I looking for?
- **Key:** What type of information do I advertise having?
- **Value:** What actual information do I provide when asked?

The complete attention mechanism combines all three:

- Use query-key similarity scores to find who's relevant
- Convert these scores to attention weights (how much to listen to each person)
- Combine values according to these weights to get your final information

Bringing it All Together

Let's see it in action with another example:

When processing "bank" in "I went to the bank to deposit money":

1. "Bank" sends out its query: "What kind of entity am I?"
2. "Deposit" and "money" have keys that strongly match this query
3. The values from "deposit" and "money" contain financial context information
4. "Bank" updates its representation by blending these values according to the attention weights

The result? "bank" now carries financial context, distinguishing it from a riverbank.

Here we have described the core idea underneath the attention mechanism of the transformer architecture in terms of the query, key, and value vectors associated with each token.

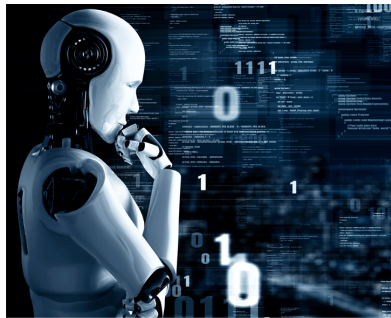
Self-Attention vs. Cross-Attention

What we've described—words attending to other words in the same sequence—is called **self-attention**. But the same mechanism works when queries come from one sequence and keys and values come from another. This **cross-attention** is how models handle language translation ("chat" in French text attending to "cat" in English), question answering (question words attending to document words), and many other tasks that involve relating different pieces of text.

2.7 Pre-training Architecture and Objectives

The Power of Self-Supervised Learning

We have described the core idea of attention in the transformer model architecture. However, we have not yet explained how the query, key, and value extraction matrices (W_q, W_k, W_v) and the initial embedding vectors (e_i) are learned. Before training, these model parameters are initialized to random values. Running text through even the most sophisticated transformer architecture with random



parameters would be useless—the resulting attention scores would be meaningless, and the language model would generate nothing but gibberish.

So how do large language models learn these critical parameters? Language models employ an elegant self-supervised learning approach: They become their own teachers. Instead of requiring millions of human-annotated examples, they learn from regular text by hiding portions of it and learning to predict what's missing. This self-supervision objective is quantified as a loss quantifying the

misprediction by the model under the current parameter values. Backpropagation iteratively updates the parameter values to minimize this prediction error. This self-supervised approach allows using the entire text on the internet as a vast training dataset to be exploited by the models to learn rich representations of language structure, semantics, and world knowledge.

Two Paths to Understanding Language

There are three fundamental approaches to hiding text for training language models, each with its own transformer architecture and learning objective.

Masked language modeling (MLM) works like a fill-in-the-blank exercise. The model randomly hides tokens throughout a sentence and learns to predict them using context from both directions. Given "The cat sat on the [MASK], by the fireplace" the model can look both backward to "cat" and "sat" and forward to "by" and "fireplace" to deduce the missing token is likely "mat" or "floor." This bidirectional visibility is typically achieved with **encoder-only architectures** where every token can attend to every other token freely. Models such as [BERT](#), [RoBERTa](#), and [ELECTRA](#) exemplify this approach. Because these models see the full context simultaneously, they build rich representations ideal for tasks such as classification, question answering, and sentiment analysis—in other words, in situations where understanding matters more than generation. However, MLM models are generally not used directly for text generation.

Causal language modeling (CLM), also referred to as autoregressive modeling or next token prediction task, takes a different approach: It predicts the next word given all previous words. The model processes text left-to-right. Given "The cat sat," it predicts "on"; given "The cat sat on," it predicts "the." This sequential prediction is typically implemented with **decoder-only architectures** with masked self-attention—each token can only see backwards, never forwards. The GPT series, LLaMA, and Claude

follow this pattern. This architecture has become dominant because it naturally supports text generation tasks—the training objective of next-token prediction aligns perfectly with how the model is used at inference time. This alignment also explains why CLM models tend to scale more predictably than bidirectional approaches whose training objectives often differ from their downstream inference tasks.

Span corruption is another important pretraining objective similar to MLM, which masks out contiguous spans—or chunks—of tokens rather than individual tokens scattered randomly. Instead of predicting single missing words, the model learns to reconstruct whole missing phrases based on the surrounding context. This approach encourages understanding of longer-range dependencies and richer context. Span corruption is commonly used in encoder-decoder architectures such as [T5](#), where the encoder processes the corrupted input bidirectionally, and the decoder autoregressively generates the missing spans. By combining bidirectional encoding with autoregressive decoding, span corruption bridges the gap between MLM’s focus on understanding and CLM’s strength in generation.

The Scale of Modern Pre-Training

Modern LLM models are trained on a staggering scale of data: GPT-3 was trained on roughly 300 billion tokens, LLaMA 2 on about 2 trillion, and GPT-4 is estimated to have processed over 10 trillion tokens—more text than a human could read in thousands of lifetimes. The data comes from large-scale, publicly available corpora: filtered web pages, digitized books, research papers from repositories such as arXiv, curated encyclopedic sources, and large collections of source code. While such sources vary in trustworthiness, training pipelines apply extensive filtering and deduplication to improve quality.

Training generally progresses in phases, measured as percentages of total training steps (e.g., 300,000 - 1,000,000+ for different Llama2 models):

- **Early phase (≈0–20%):** The model learns low-level patterns—how characters form words, where spaces and punctuation appear, and frequent short-word sequences.
- **Middle phase (≈20–70%):** Grammatical and syntactic competence emerges, including subject-verb agreement, tense consistency, and maintaining topical coherence.
- **Late phase (≈70–100%):** Higher-order abilities appear—understanding analogies, following multi-step reasoning, and synthesizing information across domains.

Emergent Abilities and Foundation Models

As models scale, they don't just get better at what they already do—they develop entirely new capabilities. At around 1 billion parameters, few-shot learning emerges: Show the model a few examples, and it generalizes to new cases. At around 10 billion parameters, complex reasoning appears: multi-step problem solving, following logical chains, understanding analogies. Beyond 100 billion parameters, we see sophisticated abilities such as writing functional code, solving mathematical problems, and engaging in nuanced dialogue.

These jumps aren't gradual—they're phase transitions, like water suddenly becoming ice. A model that completely fails at arithmetic at one scale might solve complex word problems at the next. This emergence of capabilities from scale alone remains one of the field's most fascinating mysteries.

The result is what we call foundation models: general-purpose systems that understand language broadly and can be adapted for countless specific tasks. They are versatile tools with many capabilities built in, waiting to be activated through fine-tuning or clever prompting. The same model that writes poetry might be able to debug code, answer science questions, and engage in creative storytelling, all without task-specific training.



Pause and Reflect: Consider how different pre-training objectives (MLSM vs. CLM) might affect what the model learns. Why might a model trained to predict the next word be better at generating stories than one trained to fill in blanks? What advantages might the blank-filling model have for other tasks?

2.8 How Generative AI Models Learn

From Random to Recognizable

When we initialize any generative model—whether it's a language model, image generation model, or any other generative system—it knows absolutely nothing. A transformer's weights are random numbers that would produce gibberish. Yet somehow, through training, these random initializations transform into systems that can write coherent text, generate realistic images, or create other meaningful outputs. How does this remarkable transformation happen?

The answer lies in a fundamental statistical inference principle: **maximum likelihood estimation (MLE)**. The core insight is that we treat our model as a probabilistic data generator and train it to assign high probability to observed data—whether that's a corpus of high-quality text from the internet or a collection of images. In other words, we learn model parameters θ^* such that the model's probability of generating the training data is highest.

Mathematically:

$$\theta^* = \operatorname{argmax}_{\theta} \prod_{x \in \mathcal{D}} p_{\theta}(x),$$

<Alternative text: $\theta^* = \operatorname{argmax}_{\theta} \prod p_{\theta}(x)$ >

where $\mathcal{D}$ is the training data and $p_{\theta}(x)$ is the probability of generating a piece of text (or image) x present in $\mathcal{D}$. The mathematical framework ensures that a model that successfully explains the training data will also generate new samples that are statistically similar to the training distribution.

But there's a catch: **The simplest way for a model to maximize likelihood would be to perfectly memorize the training data and regurgitate it verbatim.** While this would make generating the training data highly probable, even certain, it would be neither interesting nor useful—we want models that learn underlying patterns and generate novel, meaningful content. How do we encourage pattern learning over memorization? Machine learning employs one or more of the following approaches:

1. Introduce **controlled randomness (noise)** in the model making it impossible to output the training data as is.
2. Leverage **compression** by keeping fewer parameters in the model than what is required for complete memorization. Even though modern models have billions of parameters, the training data often contains trillions of tokens. This compression bottleneck makes memorization impossible, forcing the model to discover efficient representations of underlying patterns.
3. **Regularization** techniques act as a learning bias toward simplicity. Rather than fitting complex, memorization-based solutions, regularization encourages the model to find simpler, more generalizable patterns. Some regularization approaches include stopping the training early, preventing the parameters from getting too fine-tuned; constraining the weights to not take extreme values; and finally, using dropout, by randomly turning off neurons during training by setting their outputs to zero, effectively preventing the network from becoming overly dependent on specific pathways and forcing it to learn more robust, generalizable representations.

The maximum likelihood approach described above relies on assigning probabilities to a sentence (or an image). But there's an even deeper philosophical issue here. What does it even mean to assign a probability to a sentence like "I don't want to use tahini in the recipe tonight"? This isn't like flipping coins or rolling dice, where the probability space is clear. Language is creative and infinite—we constantly produce valid sentences that have never been uttered before in human history.

This troubled Noam Chomsky so much that he famously declared, "It must be recognized that the notion of 'probability of a sentence' is an entirely useless one, under any known interpretation of this term." He argued that grammaticality (whether a sentence follows the rules of language) and probability are orthogonal concepts. The sentence "Colorless green ideas sleep furiously" is grammatically perfect but semantically nonsensical and would have probability zero in any corpus. Meanwhile, common ungrammatical utterances like "Me want cookie" would have higher probability but violate grammatical rules.

So, what are language models actually learning when they maximize likelihood? They're not learning the "true" probability of sentences—that may not even exist. Instead, they're learning a distribution that assigns high probability to text that resembles their training data. This is both more limited and more powerful than it might first appear:

- **More limited:** The model can only capture patterns present in its training data. It has no access to the infinite creativity of human language or the grammatical rules Chomsky championed.
- **More powerful:** By learning from massive, diverse corpora, the model captures not just grammar but also style, register, topic coherence, and countless subtle patterns that would be impossible to manually encode.

The Power of Next-Token Prediction

Consider what the model must learn to maximize this likelihood on internet-scale text:

Given "The cat sat on the ___", predicting "mat" correctly requires:

- **Syntax:** Understanding that determiners precede nouns
- **Semantics:** Knowing cats sit on objects
- **World knowledge:** Recognizing mats as common surfaces
- **Long-range context:** Earlier mentions might disambiguate

Every prediction teaches something about language structure and world knowledge. Multiply by trillions of examples, and knowledge accumulates remarkably.

The Pre-Training Journey

Models see staggering amounts of text: GPT-3 trained on ~300 billion tokens, LLaMA 2 on 2 trillion, GPT-4 on 10 trillion or more. The training progresses through distinct phases:

Early Training (0-20%)	Middle Training (20-70%)	Late Training (70-100%)
The model learns token frequencies and basic patterns. Loss drops rapidly as it discovers that "the" is common, that spaces separate words, and that sentences end with periods. Generated text is repetitive and ungrammatical but shows emerging structure.	Grammar solidifies. The model learns syntactic patterns, maintains tense consistency, and develops topical coherence. The loss curve flattens as easy patterns are mastered, but the model steadily accumulates world knowledge.	Deep capabilities emerge. The model can maintain long-range coherence, perform reasoning, and synthesize knowledge across domains. Small improvements in loss translate to dramatic capability jumps—the difference between GPT-2 and GPT-3 was largely just scale. However, with versions 4 and 5 of GPT, different types of knowledge have been shown to provide more power as the models are taken through the pre-training journey, as the following image shows.

From Generalization to Personalization

The transition from pre-training to fine-tuning mirrors the trajectory of human linguistic development: Individuals acquire general grammatical competence early in life, and they later refine their language use to suit specific contexts and domains. In computational terms, this progression can be summarized as follows:

1. **Random initialization:** Model parameters are initialized without prior knowledge.
2. **Pre-training via maximum likelihood estimation:** The model learns general linguistic and world knowledge from large-scale corpora.
3. **Fine-tuning or PEFT:** The model is adapted to perform specialized tasks with high accuracy and contextual relevance.

2.9 Data Ingestion and Storage Strategies

Introduction to Embeddings: Tokens, Vectors, and Meaning

To understand how language models process and represent text, it's important to start with three foundational concepts: tokens, vectors, and embeddings.

Tokens

A token is the smallest unit of text that a language model processes. This could be a word, a subword, or even a character, depending on the model's tokenizer. For example, the sentence "I'm happy" might be broken into tokens like "I," "'m," and "happy."

Vectors

A vector is a list of numbers—often high-dimensional—that represents a token or a larger unit of text in a format a model can understand. Vectors capture patterns and relationships that can be used for computation.

Embeddings

An embedding is a vector representation of a token, a word, or even an entire sentence or document. These vectors are generated so that similar meanings are mapped to nearby points in space. For example, the words "happy" and "joyful" will have similar embeddings, meaning their vectors will be close to each other in the model's learned space.

Embeddings are powerful because they allow language models to capture and reason about the semantic relationships between pieces of text. They serve as the foundation for downstream tasks such as classification, retrieval, translation, and more.

By defining these terms up front, we can now build on them throughout this module when discussing how language is encoded, compared, and leveraged within AI systems.

The Digital Library Metaphor

Building AI systems is like constructing a vast digital library. But unlike traditional libraries with carefully catalogued books, our library must organize an endless stream of websites, documents, conversations, images, and code. Imagine trying to organize every piece of text on the internet, plus private documents, plus real-time updates—all while making everything instantly searchable. This is the challenge of data ingestion and storage for AI.

The choices we make about what to collect, how to process it, and where to store it fundamentally shape what our AI systems can do. It's the difference between a well-organized research library and a chaotic warehouse of randomly piled books.

Where AI Training Data Really Comes From

When we talk about training data for large language models, the numbers are staggering—trillions of tokens, representing a significant fraction of all human-written text available digitally. But where does it all come from?

The Common Crawl Foundation

Common Crawl is like a time machine for the web, regularly downloading and archiving billions of web pages. But raw web data is messy—full of navigation menus, advertisements, duplicate content, and spam. The journey from raw crawl to training data involves sophisticated filtering pipelines.

The Literary Heritage

Books represent humanity's long-form thinking—novels that explore human nature, textbooks that systematically explain concepts, reference works that organize knowledge. Efforts such as Project Gutenberg have digitized thousands of out-of-copyright works. Modern models also train on more recent books, though this raises complex copyright questions the field is still grappling with.

The Academic Corpus

Papers from arXiv, PubMed, and other repositories provide rigorous, peer-reviewed knowledge. These sources are particularly valuable because they're dense with factual information, careful reasoning, and proper citations. They teach models how to think systematically and cite sources.

The Code Repositories

GitHub and similar platforms provide billions of lines of code in every programming language. This isn't just about teaching models to code—programming languages are formal, logical systems that help models learn structured thinking. Comments in code also provide a unique dataset where natural language explains formal operations.

The Structured Knowledge

Wikipedia isn't just an encyclopedia—it's a carefully structured knowledge graph. Every article links to related concepts, follows consistent formatting, and includes metadata. Wikidata goes further, providing millions of facts in structured formats: (Paris, capital of, France), (Einstein, born in, 1879).

The Proprietary Goldmines

Companies sit on treasure troves of specialized data—customer support conversations, technical documentation, product databases. This data is often higher quality than public sources because it's been created for specific purposes and validated through use.

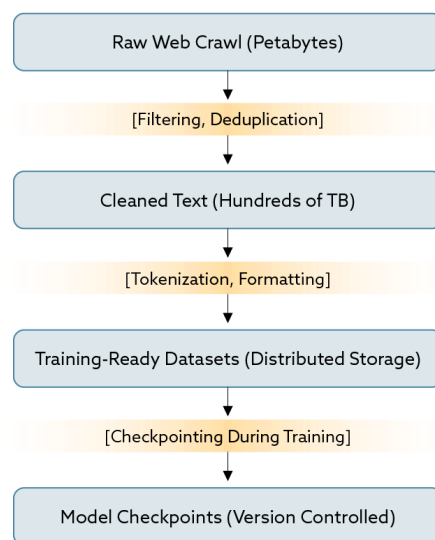
The Art and Science of Data Quality

Not all data is created equal. The challenge isn't finding more data—it's finding good data and preparing it properly.

- **Deduplication:** The internet is surprisingly and significantly redundant. The same news article might appear on hundreds of sites. Popular quotes, boilerplate text, and templated content appear millions of times. Training on duplicates wastes computation and can cause models to memorize rather than generalize. Modern deduplication uses sophisticated hashing techniques to find near-duplicates, not just exact matches.
- **Quality Filtering:** How do you automatically distinguish a thoughtful blog post from SEO spam? Researchers have developed numerous heuristics:
 - Language complexity metrics (perplexity scores)
 - Repetition ratios (spam tends to repeat keywords)
 - Document structure (well-written text has coherent paragraphs)
 - Citation patterns (quality content references sources)
- **Format Standardization:** A Wikipedia article, a PDF paper, and a GitHub repository all contain text but in vastly different formats. Extraction pipelines must handle HTML, PDF, Markdown, LaTeX, and dozens of other formats, preserving the important structure while removing formatting artifacts.
- **Language Detection and Separation:** The web speaks every human language, often mixing them within single documents. Proper language detection ensures models can be trained on specific languages or multilingual combinations intentionally rather than accidentally.
- **Privacy Protection:** Training data might include personally identifiable information (PII) such as social security and credit card numbers, names, addresses, phone numbers, among others. Privacy protection frameworks for this information vary significantly between legal bodies.

Storage Architectures for the AI Era

The storage systems for AI look nothing like traditional databases. They're not storing neat rows of structured data—we're handling massive blobs of text, embeddings in high-dimensional space, and relationships between billions of documents.



<Alternative text: A raw web crawl goes through filtering and deduplication to create cleaned text. The clean text is tokenized and formatted to create a training-ready dataset. That dataset, combined with checkpointing during training, creates version-controlled model checkpoints.>

Each stage requires different storage strategies. Raw data lives in object storage (like S3)—cheap, durable, and scalable. Active training data moves to high-performance distributed file systems that can feed hungry GPUs. Model checkpoints need version control and quick access for experimentation.

The Vector Database Revolution: Traditional databases excel at exact matches and structured queries. But AI applications need to find semantically similar content—i.e., documents "about" a topic, even if they never use those exact words. Vector databases solve this problem by storing and indexing high-dimensional embeddings.

Imagine trying to organize points in 768-dimensional space so you can quickly find the nearest neighbors to any query point. This is the fundamental challenge vector databases solve. Different indexing strategies make different trade-offs:

- **HNSW (Hierarchical Navigable Small Worlds):** Builds a graph structure for fast approximate search. Like having multiple maps at different zoom levels.
- **IVF (Inverted File Index):** Clusters vectors and searches only relevant clusters. Like organizing books by general topic before detailed search.
- **LSH (Locality Sensitive Hashing):** Uses special hash functions where similar items hash to similar values. Like a filing system where related documents naturally end up in nearby folders.

The Hybrid Reality: Production systems rarely use just one storage type. Instead, they bring in each component for its strengths, orchestrated to provide fast, relevant results. A typical architecture might include:

- Elasticsearch for full-text search and filtering
- Vector database for semantic similarity
- PostgreSQL for structured metadata
- Redis for caching frequent queries
- S3 for document storage

Best Practices Born From Experience

Building data systems for AI isn't just about technology—it's about processes that ensure quality, reliability, and usability over time.

Version Everything

Data changes, processing pipelines evolve, and requirements shift. Version control isn't just for code:

- Dataset versions (what documents were included?)
- Processing versions (how were they cleaned?)
- Index versions (what embedding model was used?)
- Schema versions (how is metadata structured?)

Plan for Incremental Updates

You can't rebuild everything from scratch when new documents arrive. Instead, design systems that can:

- Add new documents without reprocessing everything
- Update embeddings when better models become available
- Remove documents for legal or quality reasons
- Track provenance: Where did each piece of data come from?

Monitor Quality Continuously

Data quality degradation is insidious. Set up monitoring for:

- Retrieval relevance (are searches finding good results?)
- Coverage metrics (what queries return no results?)
- Freshness indicators (how recent is retrieved content?)
- Usage patterns (what are users actually searching for?)

Design for Privacy

Build privacy protection in from the start:

- Access controls at the document level
- Audit logs for compliance
- Right to deletion (GDPR compliance)
- Separation of public and private data

The goal is to build a system that's not just functional today but can evolve with your needs, like a library that can grow, reorganize, and adapt while never losing a book.



Pause and Reflect: Consider the ethical implications of data collection and storage for AI. Who decides what data is included in training sets? How do we balance the benefits of comprehensive data with privacy rights? What responsibilities do we have when building systems that can access and synthesize vast amounts of information?

2.10 Retrieval-Augmented Generation (RAG) Deep Dive

Why RAG Transforms AI Capabilities

Language models, for all their sophistication, are trapped in the moment their training ended, unable to learn about events that happened even one second later. They're like historians who stopped reading at a certain date—brilliant about everything before that moment, but completely unaware of anything after.

This creates four fundamental problems that RAG elegantly solves:

- **The Knowledge Cutoff Problem:** Unlike RAG, LLMs lack modularity, making updates more costly. While RAG allows quick replacement of documents in its database, updating an LLM typically requires more time and resources—potentially limiting how frequently the model can be updated.
 - For example, when GPT-4's training ended in early 2024, the model became closed off from learning about subsequent events. It doesn't know who won the 2024 Olympics or what happened in yesterday's stock market. RAG breaks through this temporal barrier by retrieving current information.
- **The Hallucination Problem:** When LLMs don't know something, they often generate plausible-sounding but false information. It's not malicious—it's more like a student who, unsure of an answer, writes something that sounds reasonable. RAG reduces hallucinations by providing actual sources, grounding responses in retrieved facts, thus reducing hallucinations in the output.
 - Note that RAG may still blend retrieved facts with model-internal knowledge, and it requires careful validation to mitigate the hallucination problem. Hallucinations result from the probabilistic nature of the model, where it optimizes for fluency and coherence.
- **The Specialization Problem:** A general-purpose LLM knows a little about everything but may lack deep expertise in specialized domains. It's like expecting a general practitioner to perform neurosurgery. RAG can tap into specialized databases, technical manuals, and domain-specific knowledge bases.
- **The Privacy Problem:** Public LLMs can't access your company's internal documents, your personal notes, or proprietary databases. RAG creates a bridge to private knowledge while maintaining security boundaries.

The RAG Architecture

RAG works by combining the reasoning power of LLMs with the precision of information retrieval. It's not just about fetching documents. It's about creating an intelligent pipeline that understands questions, finds relevant information, and synthesizes coherent answers.

Here's how it works in practice.

Step one is document processing. Before any questions are asked to your LLM your documents go through extensive pre-processing. PDFs are passed, any websites are scraped, databases are indexed. Each document is then broken into chunks, perhaps paragraphs or sections, small enough to be relevant, but large enough to be coherent.

Each chunk is then converted into an embedding. A high dimensional vector that captures its semantic meaning. We can think of this as creating a unique fingerprint for each piece of text. Similar concepts get similar fingerprints even if they use different words.

Step two, query understanding. When you ask, what was our third quarter revenue? The system must understand that you're looking for financial information from a specific time period. Your question is converted into the same embedding space as the documents. Now we can find documents that are semantically close to your question.

Step three is intelligent retrieval. The challenge is that documents don't contain questions. They contain statements, figures, and narratives. The retrieval system must bridge this gap. Finding passages most likely to contain your answer.

Modern RAG systems often use a hybrid search. They use a semantic search which finds conceptually related content, revenue matches, earnings, income and sales. They also use a keyword search which ensures specific terms aren't missed. Third quarter 2024 exact product names.

Additionally, metadata filtering narrows results by date, department, or document type. The system might retrieve. According to the third quarterly 2024 financial report total revenue reached \$4.7 billion. Representing a 23% year on year increase.

Step four is context integration and answer generation. The retrieved documents now become contexts for the LLM. The system will take all the retrieved documents and rank the passages by relevance. Removing redundancy and preserving important relationships, and then fit this within the model's context window.

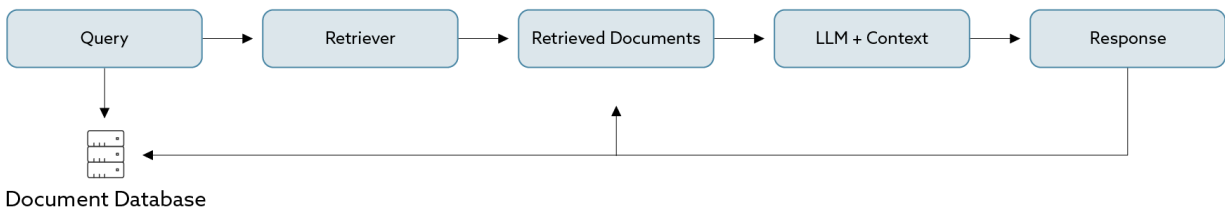
Finally, **step 5**, the large language model will synthesize an answer using both its trained knowledge and the retrieved context. It's not just copying, it's understanding, reasoning and explaining. Based on our third quarter financial report revenue was \$4.7 billion. Up 23% from last year. The growth was primarily driven by strong cloud services performance which offset weakness in our consumer division.

RAG Example: The Question

Now, let's trace the journey of a question through a RAG system. The question is: "What was our Q3 revenue?" Here's what happens behind the scenes:

First, your question enters the retrieval pipeline. But here's the challenge: Your documents aren't organized by questions—they're full of statements, figures, and narratives. The system must somehow find the passages most likely to contain your answer. This is like looking for a needle in a haystack, except you're not even sure what the needle looks like.

The solution is to transform both your question and all documents into a mathematical space where semantic similarity becomes geometric proximity. Imagine every possible concept floating in a vast multidimensional space. "Revenue" might be close to "earnings" and "income" but far from "expenses" or "headcount." By converting text to coordinates in this space—i.e., embeddings—we can find documents that are conceptually near your question.



<Alt text: The journey of a question through a RAG system. A query enters the retrieval pipeline, and documents are retrieved and passed to the LLM. Then a response is generated.>

But retrieval is just the beginning. Once we've found relevant documents, we face a new challenge: How do we help the LLM use this information effectively?

The Art of Context Integration

Finding relevant documents is like bringing reference books to an exam—they are helpful only if you know how to use them. The LLM must somehow incorporate retrieved information into its response while maintaining coherence and accuracy.

This is where prompt engineering becomes crucial. We construct a prompt that clearly delineates retrieved context from the user's question:

Context	Question	Answer
According to the Q3 financial report, total revenue for the third quarter was \$4.7 billion, representing a 23% year-over-year increase. The growth was primarily driven by strong performance in the cloud services division.	What was our Q3 revenue?	Based on the Q3 financial report, our third quarter revenue was \$4.7 billion, which represents a 23% increase compared to the same quarter last year.

The LLM acts as an intelligent synthesizer, weaving retrieved facts into coherent, contextual responses. It's not just copying—it's understanding and reformulating.

The Evolution of Retrieval Methods

The retrieval landscape has evolved dramatically, offering different tools for different needs:

Sparse Retrieval (BM25)

The traditional approach treats documents as bags of words, scoring relevance based on term frequency and document frequency. It's like a meticulous librarian who counts how often specific words appear. When you search for "machine learning," it finds documents with those exact terms. It may be fast and

predictable, but it misses semantic connections—it wouldn't know that "deep learning" and "neural networks" are related concepts.

Dense Retrieval

The modern approach uses neural networks to understand meaning. Documents and queries become points in semantic space. Now, "machine learning," "ML," "artificial intelligence," and "deep learning" cluster together because the model understands their conceptual relationships. It's like having a librarian who actually understands the content, not just the words.

Hybrid Approaches

Hybrid approaches offer the best of both worlds. They use sparse retrieval for precise term matching (great for finding specific product names or technical terms) and dense retrieval for conceptual understanding. Then, they re-rank results using a more sophisticated model. It's like having both a detail-oriented assistant and a subject matter expert working together.

Multimodal Retrieval

Multimodal retrieval is the process of retrieving relevant information across different data types (modalities) such as text, images, audio, video, or code by mapping them into a shared semantic space where cross-modal similarity can be measured.

Advanced RAG Techniques

As RAG systems have matured, they've developed sophisticated strategies that go beyond simple retrieve-and-answer:

- **Multi-Hop Retrieval:** Sometimes answering a question requires following a chain of information. Ask "Who is the CEO of our biggest competitor?" and the system might first need to retrieve documents identifying your biggest competitor, then search for information about their CEO. It's like solving a mystery where each clue leads to the next.
- **Hierarchical Indexing:** Not all text chunks are created equal. A single document might contain high-level summaries, detailed sections, and specific data points. Hierarchical indexing organizes information at multiple granularities:
 - Document level: "2023 Annual Report"
 - Section level: "Financial Performance"
 - Paragraph level: "Q3 Revenue Analysis"
 - Sentence level: "Revenue increased 23% to \$4.7B"
 - The system learns to retrieve at the right level of detail for each query.
- **Query Expansion:** Users don't always ask perfect questions. Someone might ask "How much did we make?" when they mean "What was our total revenue last quarter?" Query expansion uses the LLM to interpret intent and generate multiple search queries, casting a wider net for relevant information.
- **Chunking Strategy:** The way documents are divided into chunks—whether by fixed size or semantic boundaries—greatly influences what content is retrieved and how well it aligns with user queries.

- **Vector Store Examples:** After generating embeddings, they need to be stored in specialized databases (vector stores), such as FAISS or Pinecone, which allow for efficient similarity-based retrieval.
- **Token Limit Constraints:** Since LLMs can only process a limited number of tokens per input, the amount of retrieved content passed into the model must be carefully managed to stay within these bounds.
- **Post-Retrieval Ranking (Reranking):** Not all retrieved results are equally relevant, so reranking techniques help prioritize and select the most contextually useful documents before passing them to the model.



Pause and Reflect: Consider how RAG changes the nature of AI Applications. Instead of trying to train models on every possible piece of information, we can keep models general and give them access to specific knowledge as needed. What implications does this have for privacy, for keeping information current, and building specialized AI tools?

Retrieval Augmented Generation fundamentally changes what's possible with LLMs. It allows them to be always current, connected to live databases, news feeds or streaming data.

They are also infinitely specializeable. Add your industry's technical manuals, regulations, and best practices and you can create domain expert LLMs on demand. Additionally, they are also enterprise ready. You may keep sensitive data private while still leveraging AI.

They are also scalable. Instead of retraining massive LLMs on your documents just add them to your RAG index, there is a linear cost and immediate results. Real companies are using RAG for customer support for instantly searching product documentation. Also used for legal research, in finding relevant cases across millions of documents. For assisting medical diagnosis, accessing the latest research and financial analysis. Combining market data with internal reports and even code generation. Retrieving relevant examples from private repositories.

The advantage with RAG is that you get the best of both worlds. The linguistic sophistication of large language models and the precision of your specific data. It's not about replacing human expertise. It's about augmenting it with an AI that has perfect recall of everything your organization knows.

RAG transforms LLMs from impressive demos into practical tools that solve real business problems. It's the bridge between the promise of AI and its practical application in a world where knowledge is vast, specialized, and constantly changing.

2.11 Knowledge-Augmented Generation (KAG)

Beyond RAG: Structured Knowledge Integration

RAG treats information as documents—chunks of text to be retrieved and presented. But human knowledge isn't just text; it's a rich web of entities, relationships, and rules. When you know that Paris is the capital of France, you're not just storing a sentence—you're recognizing entities (Paris, France) and their relationship (capital of).

Knowledge-augmented generation (KAG) represents a fundamental shift in how we augment language models. Instead of retrieving text that might contain an answer, we query structured knowledge that provides precise facts. It's the difference between giving someone a book about France and directly telling them "Paris is the capital, population 2.2 million, founded in the 3rd century BC."

The Knowledge Graph Advantage

Imagine two assistants helping you plan a trip. The first (RAG) runs to the library and brings back travel guides, blog posts, and articles about your destination. The second (KAG) has memorized facts such as flight connections, hotel locations, tourist attractions, and their relationships—this hotel is near that museum, which is open on these days, when this festival happens.

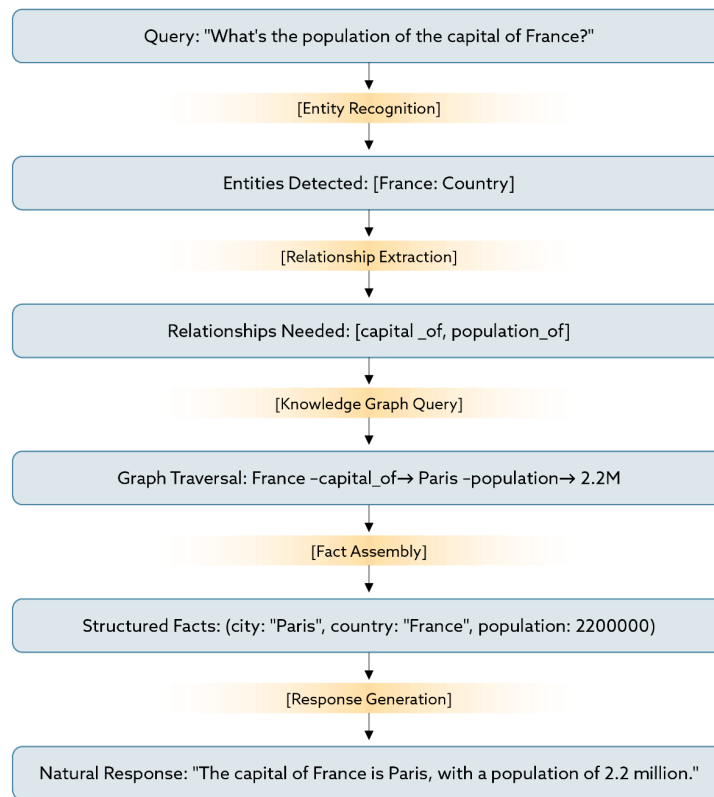
Both are helpful, but they offer fundamentally different kinds of assistance. The RAG assistant provides rich, contextual information but requires you to extract specific facts. The KAG assistant provides precise, structured information but might miss nuanced insights that emerge from narrative text.

This is why modern systems increasingly combine both approaches. KAG excels at:

- **Factual Precision:** When you need exact information—populations, dates, scientific constants—knowledge graphs provide authoritative answers without the ambiguity of natural language.
- **Multi-Hop Reasoning:** Complex questions often require following chains of relationships. "Who is the CEO of the company that makes the iPhone?" requires knowing Apple makes the iPhone, then finding Apple's CEO. Knowledge graphs make these inference chains explicit and traceable.
- **Consistency Enforcement:** In a knowledge graph, Paris can't simultaneously be the capital of France and Germany. Relationships follow logical rules that prevent contradictions.
- **Real-Time Updates:** Changing a fact in a knowledge graph immediately updates all queries that depend on it. Update the current president, and every related query reflects the change.

The Architecture of Knowledge

Let's trace how KAG processes a complex query. Here's the journey when someone asks "What's the population of the capital of France?":



<Alternative text: A query, "What's the population of the capital of France?" passes through entity recognition. Entities are detected; in this case, the country of France is identified. The model then extracts relationships between parts of the query, such as what it means for a city to have a capital or population. This is passed to a knowledge graph query which connects France, "capital_of," Paris, "population," and 2.2M. This is assembled into structured facts, which are then generated into a response. >

Each step transforms unstructured language into structured operations, executes precise lookups, then transforms it back into natural language. It's like having a universal translator between human questions and database queries.

The table below summarizes some of the differences between RAG and KAG across various dimensions.

Dimension	RAG (Retrieval-Augmented Generation)	KAG (Knowledge-Augmented Generation)
Knowledge Access	Dynamic retrieval at runtime from external sources	Pre-integrated or statically embedded knowledge
Latency	Higher latency due to external retrieval steps	Lower latency (no external lookups at inference)

Compute Load	More compute-intensive at inference (retrieval and generation)	More compute-efficient at inference
Adaptability	High adaptability; updates via index refresh	Lower adaptability; requires manual updates to integrated knowledge
Relevance Quality	Variable—depends on retrieval quality (may include noisy results)	High precision—curated or scoped context improves quality
Scalability	Scales well with changing or large knowledge bases	Less scalable—harder to manage large or dynamic knowledge sources
Maintenance	Requires maintaining retrieval infrastructure (e.g., vector DBs)	Requires curating and integrating knowledge manually
Use Case Suitability	Better for real-time, open-domain, or up-to-date information needs	Better for domain-specific, stable knowledge tasks

Knowledge Representation: The Foundation

The power of KAG depends on how we represent knowledge. Different representations enable different capabilities.

Triple Stores:

This is the simplest and most flexible format. Each triple captures one atomic fact. Simple queries combine multiple triples. Complex reasoning follows paths through the graph. The elegance lies in the simplicity—any fact can be expressed as subject-predicate-object.

- (Paris, is_capital_of, France)
- (Paris, has_population, 2200000)
- (Eiffel_Tower, located_in, Paris)
- (Eiffel_Tower, height_in_meters, 330)

Property Graphs:

Property graphs are richer representations where both nodes and edges have properties:

- Node: {id: "Paris", type: "City", founded: "3rd century BC"}
- Edge: {from: "Paris", to: "France", type: "capital_of", since: "987 AD"}

This enables temporal reasoning (When did Paris become the capital?), uncertainty representation (confidence scores on facts), and metadata tracking (source, last updated).

Ontologies:

Ontologies are formal specifications of what types of entities and relationships are allowed: For example, cities can be capitals of countries (but not of people), people can be born in places (but places can't be born in people), and if A is north of B, then B is south of A.

As a result, ontologies reduce the chance of logical inconsistency associated with entities labeled as facts and enable logical inference.

Integration Strategies: Weaving Knowledge Into Language

The real magic happens when structured knowledge enhances natural language generation. KAG systems use several strategies:

Fact Injection

Augment the prompt with relevant facts:

User Query	System Query	Enhanced Prompt	LLM Response
"Tell me about Paris"	System Query: MATCH (p:City {name: "Paris"}) RETURN p.population, p.country, p.founded	"Tell me about Paris [Population: 2.2M, Country: France, Founded: 3rd century BC]"	"Paris, the capital of France, is a city of 2.2 million people with a rich history dating back to the 3rd century BC..."

Reasoning Chains

Make multi-step inferences explicit:

User Query	System Reasoning	LLM Response
"Is the Eiffel Tower taller than Big Ben?"	• Eiffel Tower height: 330m • Big Ben height: 96m • 330 > 96: True	"Yes, the Eiffel Tower at 330 meters is significantly taller than Big Ben at 96 meters."

Constraint Satisfaction

Ensure generated content respects known facts:

Generation Prompt	Constraints From KAG	LLM Response
"A story about Napoleon"	• Born: 1769, Corsica • Died: 1821, Saint Helena • Cannot appear in scenes after 1821 • Was not very tall (1.68m)	The LLM generates creative content while respecting historical facts.

The Challenges and Opportunities

KAG isn't without challenges. Building and maintaining knowledge graphs requires significant effort. Entity disambiguation remains difficult (which Paris? The city or Paris Hilton?). Combining symbolic reasoning with neural generation sometimes produces awkward handoffs.

But the opportunities are immense. Imagine AI assistants that never confuse facts, that can trace their reasoning, and that update their knowledge instantly; systems that combine the creativity of language models with the precision of databases; and applications where hallucination isn't just reduced but structurally impossible for factual queries.

KAG represents a bridge between the symbolic AI of the past and the neural AI of the present. It acknowledges that not all knowledge fits neatly into continuous vector spaces—sometimes you need discrete facts, logical rules, and structured relationships.

As we build increasingly powerful AI systems, KAG enables a path toward more reliable, interpretable, and controllable generation. It's not about replacing neural approaches but augmenting them with the kind of structured knowledge that humans have always used to organize and reason about the world.



Pause and Reflect: Consider how combining neural and symbolic approaches might change AI capabilities. What kinds of applications become possible when AI systems can access both the flexibility of language models and the precision of knowledge graphs? How might this change our relationship with AI—from tools that generate plausible text to systems that can engage in verified reasoning?

If your organization allowed genAI API usage, what data risks might arise? This ties the KAG discussion to practical concerns around data exposure, privacy, and trust when integrating external systems.

2.14 Cloud Service Models for Generative AI

The Service Model Spectrum

The differences among various cloud service models are all about balancing control against convenience. Infrastructure as a Service (IaaS) provides maximum control but requires significant expertise. Software as a Service (SaaS) offers immediate access with minimal setup but limited customization. Platform as a Service (PaaS) occupies the middle ground.

The Abstraction Trade-Off: Higher abstraction levels enable faster deployment and reduce operational overhead but decrease control and customization options.

Infrastructure as a Service (IaaS)

Maximum Control and Responsibility

IaaS provides raw computational resources without higher-level abstractions. For generative AI, this typically means high-performance instances with specialized hardware such as GPUs or TPUs. Note that TPUs are specialized hardware developed and offered exclusively by Google Cloud.

Organizations gain complete control over their software stack, from operating systems to specific model architectures. This enables optimization for specific use cases and implementation of proprietary algorithms.

The responsibilities are substantial: managing OS updates, security patches, monitoring systems, backup procedures, and disaster recovery. Organizations must implement their own model-serving infrastructure, including load balancing and auto-scaling.

Example: A financial services company might choose AWS EC2 P4d instances (computing servers) with NVIDIA A100 GPUs, install PyTorch (Python AI/ML framework), implement custom preprocessing pipelines, design neural networks optimized for financial documents, and build monitoring systems. This provides complete flexibility but requires significant ML engineering expertise.

Choose IaaS when:

- You need custom model architectures
- You have specific compliance requirements
- You want maximum infrastructure control
- You possess deep ML engineering expertise

Platform as a Service (PaaS)

Managed Development Environments

PaaS offerings provide pre-configured development and deployment environments that minimize infrastructure complexity while preserving customization flexibility. These include integrated development environments, managed training infrastructures, and automated deployment pipelines.

The service AWS SageMaker provides Jupyter environments, managed training jobs that scale from single instances to distributed clusters, automatic model tuning, and one-click deployment. Google Vertex AI emphasizes ecosystem integration, while Azure Machine Learning focuses on enterprise governance.

Example: A customer service organization might upload training data to cloud storage, use Vertex AI's managed training to fine-tune a foundation model on support conversations, evaluate results with built-in tools, and deploy to auto-scaling endpoints.

Choose PaaS when:

- You need custom models with managed infrastructure
- You want MLOps capabilities
- You have ML expertise but limited infrastructure knowledge
- You require team collaboration features

Software as a Service (SaaS)

Ready-to-Use AI Capabilities

SaaS provides immediate access to pre-trained, production-ready models through simple API interfaces. These services handle all model hosting, scaling, and maintenance aspects.

OpenAI's GPT models represent the archetypal SaaS offering. Organizations integrate sophisticated language capabilities by making HTTP requests without understanding transformer architectures or managing infrastructure. The service AWS Bedrock provides standardized access to multiple foundation models through unified APIs. Bedrock is a broker platform as it gives access to models developed by third parties like Anthropic, Meta, and others via a unified interface.

Example: A marketing agency might integrate OpenAI API into their content management system, sending article topics to GPT-4 and receiving generated content for writers to edit. The agency benefits from advanced AI without needing ML engineers or infrastructure management.

Choose SaaS when:

- You need quick time-to-market
- Standard use cases fit requirements
- You have limited ML expertise
- You want predictable costs with minimal overhead

Service Model Selection Framework

Technical expertise is crucial for selection. IaaS requires deep ML engineering and infrastructure management skills. Customization requirements influence appropriateness—novel architectures may need IaaS flexibility, while standard use cases suit SaaS offerings.

Security compliance requirements often favor IaaS or PaaS for greater control. Time-to-market pressures favor higher-level service models, as SaaS can be integrated into your application environment in days, while IaaS may require months.

2.15 Deployment Strategies and Hybrid Environments

The Deployment Landscape

AI deployment extends beyond choosing service models. Organizations must consider where workloads run, data flow management, and balancing performance, compliance, cost, and control requirements.

A deployment strategy is a comprehensive plan defining where AI workloads execute, data flow between environments, security measures, and system management across the life cycle. Modern AI deployment strategies span multiple dimensions: physical resource location, degree of control, integration requirements, and regulatory constraints. The landscape includes public clouds, private clouds, hybrid configurations, and multi-cloud strategies.

Public Cloud Deployments: Scale and Managed Services

Public cloud deployments leverage massive scale, global reach, and sophisticated managed services. They offer the fastest path to production and access to cutting-edge infrastructure that would be prohibitively expensive to build independently.

Scalability advantages are pronounced for generative AI with variable resource requirements. Training may need hundreds of GPU instances briefly, while inference may scale from zero to thousands of requests per second within minutes.

Example: A news organization implements serverless article summarization using AWS Lambda functions calling Bedrock APIs. S3 events trigger lambda functions that process articles through Claude models and store results. This scales automatically without infrastructure management.

Challenges include data sovereignty concerns, vendor lock-in risks, and potential latency issues for real-time applications.

Private Cloud Deployments: Control and Compliance

Private cloud provides dedicated infrastructure with direct organizational control, offering maximum control over security, compliance, and performance, while potentially providing cost advantages for consistent, high-volume workloads.

Control advantages are valuable for strict regulatory requirements or proprietary data that cannot be processed in shared environments. Performance predictability eliminates noisy neighbor effects that are common in shared environments.

Example: A trading firm deploys NVIDIA DGX systems for real-time market analysis, including custom models, low-latency networking, and specialized security. Private deployment ensures microsecond-level latency and complete control over sensitive algorithms.

Challenges include significant operational overhead, capital expenditure requirements, and scaling limitations compared to public cloud elasticity.

Hybrid Cloud Architecture: Balancing Requirements

Hybrid architectures combine public and private deployment advantages while mitigating disadvantages. They involve strategic workload and data placement based on specific requirements for security, performance, compliance, and cost.

Key Principles:

- **Data Gravity:** Keep sensitive data in controlled environments while processing less sensitive data where resources are available.
- **Workload Optimization:** Match workloads to environments supporting their requirements.
- **Burst Capacity:** Use public cloud for peak demands exceeding private capacity.
- **Disaster Recovery:** Leverage multiple environments for resilience.

Example: A healthcare system keeps patient data in a private cloud for HIPAA compliance while running AI training in a public cloud using de-identified data. Real-time clinical support operates on-premises while research leverages public cloud scaling.

Implementation Patterns

- **Data Residency Pattern:** Data remains in controlled environments while computation uses different locations through preprocessing and anonymization.
- **Burst Computing Pattern:** Maintain cost-effective private infrastructure for normal operations while accessing public cloud for peaks.
- **Development-Production Split:** Use public cloud for development and testing while maintaining private infrastructure for production.
- **Multi-Cloud Resilience:** Distribute workloads across multiple providers to reduce vendor lock-in and improve resilience.

2.16 Security and Interoperability

The AI Security Imperative

An AI security threat model is a framework for understanding attack vectors including direct model attacks, API vulnerability exploitation, computational resource abuse, and sensitive information extraction. Threat modeling and security attacks will be covered in depth later in the course.

Data privacy concerns extend beyond traditional security. AI models may memorize and reproduce sensitive training data, and prompts and responses often contain sensitive information, requiring careful protection. Let's explore some types of authentication and authorization strategies.

Here are the top 10 risks in 2025, ranked by OWASP:

1. Broken Access Control
2. Cryptographic Failures
3. Injection
4. Insecure Design
5. Security Misconfiguration
6. Vulnerable and Outdated Components
7. Identification and Authentication Failures
8. Software and Data Integrity Failures
9. Security Logging and Monitoring Failures
10. Server-Side Request Forgery

[You can optionally read more about the OWASP Top 10 by clicking here.](#)

API Security

Authentication Strategies

Authentication is the process of verifying the identity of a user or system. It answers the question: "Who are you?" Common authentication methods include passwords, biometrics, and security tokens.

API Keys: This is a simple but limited approach providing basic identification and tracking. It is easy to implement but typically long-lived and difficult to rotate regularly.

OAuth 2.0: This is a sophisticated framework separating authentication from authorization, enabling fine-grained scope-based permissions and token refresh mechanisms. It is valuable for enterprise identity system integration. OAuth 2.0 is an open industry standard widely adopted across providers.

JWT Tokens: This is a stateless approach to encoding user identity, permissions, and usage limits directly in tokens. It eliminates server-side session storage needs and enables efficient distributed authentication.

Authorization Strategies

Authorization is the process of determining what an authenticated user is allowed to do. It answers the question: "What are you allowed to access?" For example, after logging in (authentication), a user may be authorized to view their profile but not to access admin settings.

Role-Based Access Control (RBAC): This is a structured approach to defining roles that encapsulate permission sets. Users are assigned appropriate roles such as developer, business user, or administrator.

Attribute-Based Access Control (ABAC): This is a flexible approach to making decisions based on user, resource, action, and environment attributes. It may consider data sensitivity, model type, time of day, or network location.

Scope-based Permissions: This offers fine-grained control over AI capabilities through specific scopes such as "text-generation" or "image-analysis."

Rate Limiting and Resource Protection

Rate limiting serves multiple purposes: preventing abuse, controlling costs, ensuring fair resource allocation, and protecting against overuse.

Multi-Dimensional Rate-Limiting Approaches:

Request-based controls API calls within time windows. It's simple but may not reflect computational cost variations.

Token-based limits input and output tokens processed within periods. It offers better cost alignment since AI costs scale roughly linearly with tokens.

Compute-based measures the actual computational resources consumed. It is the most accurate but requires sophisticated monitoring.

Cost-based directly limits the monetary value of consumed resources. It is best for business alignment but requires real-time cost calculation.

Example Rate Limiting Configuration:

```
rate_limits:
  free_tier:
    requests: 100/day
    tokens: 10000/day
  pro_tier:
    requests: 10000/day
    tokens: 1000000/day
  burst: 100/minute
```

<Alternative text: Two examples of rate limits. The free tier allows 100 requests and 10,000 tokens per day. The pro tier allows 10,000 requests and 1,000,000 tokens per day.>

Implementation Patterns:

Token bucket gives each user a bucket that fills with tokens at steady rates up to maximum capacity. Requests consume tokens; insufficient tokens reject requests. It accommodates burst usage while maintaining long-term compliance.

Fixed window divides time into discrete windows tracking usage within each. It is simple but can create boundary traffic spikes.

Sliding window tracks usage over rolling periods providing more even distribution and better spike protection.

Endpoint Protection and Communication Security

Transport Security

TLS 1.3 provides essential encryption for API communications and protects data in transit, while endpoints can still alter or weaken security through configuration changes.

Certificate pinning offers additional protection against man-in-the-middle attacks by ensuring clients only accept expected certificates.

HSTS policies ensure all communications use encrypted connections to prevent accidental unencrypted data transmission.

API Interoperability and Standards

Design Standards

OpenAPI specifications offer standardized API documentation, including authentication requirements, validation rules, and error handling. Well-documented APIs reduce integration effort and help implement proper security.

RESTful principles promote consistent, predictable interfaces that are easier to secure and monitor. Statelessness and uniform interfaces align with security best practices.

Example OpenAPI Security Specification:

```
paths:
  /v1/generate:
    post:
      security:
        - ApiKeyAuth: []
      responses:
        '200':
          description: Generation successful
        '429':
          description: Rate limit exceeded
components:
  securitySchemes:
    ApiKeyAuth:
      type: apiKey
      in: header
      name: X-API-Key
```

<Alternative Text: A breakdown of security requirements for OpenAI API call, which includes an ApiKeyAuth item that includes headers such as type of key, actual value, and name of the key, along with possible responses from OpenAI API, including a successful message and an error: rate limit exceeded message.>

Key Principles for Secure System Design

Least Privilege

The principle of least privilege means giving users, systems, and processes the minimum level of access—or permissions—necessary to perform their tasks. This limits potential damage in case of a breach or misconfiguration. For example, a user who only needs read access should not be granted write or admin privileges.

Input and Output Validation

Validating input ensures that data coming into the system is clean, expected, and safe, preventing common vulnerabilities such as SQL injection and cross-site scripting (XSS). Similarly, validating output ensures that sensitive information is not inadvertently exposed and helps preserve data integrity during system interactions.

API Gateways

API gateways serve as a centralized entry point for managing, securing, and monitoring API traffic. They enforce policies such as rate limiting, authentication, and input validation, while also acting as a buffer against malformed or malicious requests. Gateways are critical in microservices and distributed architectures.

Monitoring and Logging Best Practices

Effective monitoring and logging help detect anomalies, track user behavior, and support incident response and forensic investigations. Best practices include:

- Logging security-relevant events (e.g., failed logins, permission changes)
- Ensuring logs are tamper-resistant and stored securely
- Monitoring logs continuously using automated tools
- Correlating events across systems for better visibility

CSA's AI Controls Matrix

The Cloud Security Alliance (CSA) provides a comprehensive [AI Controls Matrix](#) that maps specific security controls across the AI life cycle. This framework addresses:

- Data governance and privacy controls
- Model security and integrity
- Infrastructure protection measures
- Operational security requirements
- Compliance and audit considerations

Security Best Practices

- Implement multiple security layers rather than relying on single mechanisms
- Use strong authentication appropriate for risk profiles
- Implement comprehensive rate limits across multiple resource dimensions
- Monitor usage patterns and implement automated anomaly detection
- Maintain detailed audit logs while protecting user privacy
- Follow established API designs and security testing or addition of a line specifying the need to assure security testing as part of development practices
- Plan incident response procedures for handling security events
- Regularly review and update security controls as threats evolve
- Implement a robust change management process that tracks code changes or algorithm adjustments or versioning controls

Findings should be categorized and remediated as part of the change management process or vulnerability management process.

2.18 Summary

Congratulations on completing Module 2! In this module, you bridged theoretical understanding with practical implementation across both model architectures and service deployment strategies.

Key Concepts Mastered

Technical Architectures: You explored how transformer models enable token-to-token interactions through multi-head attention mechanisms and layer stacking. You then learned the fundamentals of probabilistic denoising processes that gradually transform noise into coherent images through forward and reverse noise application, including advanced techniques such as classifier-free guidance, and how diffusion models compare to alternative generative approaches such as variational autoencoders (VAEs) and generative adversarial networks (GANs).

Training and Learning Systems: You explored how models are trained, including specialized techniques such as supervised fine-tuning (SFT) and reinforcement learning from human feedback (RLHF). You should now understand the fundamental difference between masked language modeling (MLM), which teaches models to fill in blanks using bidirectional context, and causal language modeling (CLM), which focuses on next-token prediction using only preceding context, while grasping how both approaches achieve maximum likelihood estimation through different factorization strategies.

Knowledge Enhancement Systems: You examined the complete RAG architecture that enhances language models with external knowledge, learning to implement various retrieval methods, including sparse retrieval (BM25), dense retrieval using semantic embeddings, and hybrid approaches. You also learned how to leverage structured knowledge through knowledge graphs and triple stores for precise factual reasoning, learning entity-relationship modeling techniques and mastering the integration of symbolic reasoning with neural generation to create systems that can perform logical inference while maintaining the flexibility of language models.

Deployment and Security: You learned to distinguish between Infrastructure as a Service (IaaS) for maximum control, Platform as a Service (PaaS) for managed development environments, and Software as a Service (SaaS) for ready-to-use capabilities, while mastering selection frameworks that evaluate trade-offs based on organizational control requirements, technical expertise levels, and compliance constraints. You also explored comprehensive security implementations, including modern authentication methods such as OAuth 2.0 and JWT tokens, multi-dimensional rate limiting across requests, tokens, compute, and cost, endpoint protection through TLS encryption and input validation, and adherence to interoperability standards.

Critical Insights

Architecture Matters: You learned that transformers enable direct token-to-token communication where "every word can look at every other word using the same attention mechanism, regardless of where they appear." This architectural understanding highlights that different attention heads naturally specialize during training—one might track grammatical relationships while another focuses on semantic

similarities—enabling more targeted prompting strategies and better interpretation of model outputs.

Context Matters: You discovered how retrieval-augmented generation (RAG) and knowledge-augmented generation (KAG) fundamentally extend model capabilities beyond their training data limitations, addressing four critical problems: the knowledge cutoff problem, the hallucination problem, the specialization problem, and the privacy problem. You also saw how RAG grounds responses in retrieved sources while KAG leverages structured knowledge graphs for precise factual reasoning, fundamentally changing AI applications from systems that might hallucinate to ones that provide verifiable, source-backed information.

Quality Over Quantity: You learned that sophisticated data curation and filtering processes are more critical than raw data volume for training effective models, since not all data is created equal and the challenge isn't finding more data—it's finding good data and preparing it properly.

Looking Ahead: Your Continued Journey in Generative AI

Future-Proofing: Plan for incremental updates, version control, and quality monitoring from the beginning because AI systems evolve rapidly and require flexible architectures that can adapt to changing requirements and new capabilities. Following established API design standards and OpenAPI specifications not only reduces integration effort and improves your security posture but ensures your systems can interoperate effectively as the AI landscape continues to evolve.

Practical Applications: You can start with simple retrieve-and-generate RAG systems and progressively enhance them with advanced techniques such as query expansion, multi-hop reasoning, and hierarchical indexing. Additionally, you must implement multi-layered security with strong authentication, comprehensive rate limiting, and detailed monitoring as essential foundations for any production AI system, ensuring both functionality and protection against the unique security challenges that generative AI presents.

Strategic Implementation: You should choose a service model based on specific organizational needs—selecting IaaS for custom architectures and compliance requirements, PaaS for managed development environments with flexibility, or SaaS for rapid deployment with standard use cases—while simultaneously designing hybrid architectures that combine public and private cloud advantages through strategic workload placement. This approach allows you to keep sensitive data controlled in private environments while leveraging public cloud scaling and innovation capabilities, creating a balanced architecture that optimizes for both security and performance across your entire AI implementation.

Change Log

Version #	Approved by	Notes
20251015	HMR	Initial release